

1교시: Compose 기본 개념과 편의성



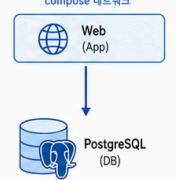
Day 5: Compose Architecture Lab

로컬에서 실행하는 5가지 Docker Compose 아키텍처 패턴

🎯 목표
 ✓ 실무에서 자주 쓰는 아키텍처 패턴을 Compose로 구성
 ✓ 서비스 간 통신, 볼륨, 네트워크를 이해
 ✓ 검증 루프로 반복하며 확실하게 익히기

1 Web + DB
두 계층 웹 애플리케이션

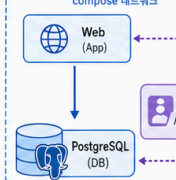
compose 네트워크



• 웹 애플리케이션과 PostgreSQL 구성
• 영속 볼륨으로 데이터 보존

2 DB 관리 UI
DB UI 도구 추가

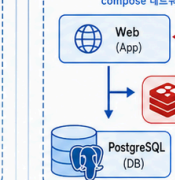
compose 네트워크



• Adminer 또는 pgAdmin으로 DB를 쉽게 관리

3 Redis Cache
캐시 계층 추가

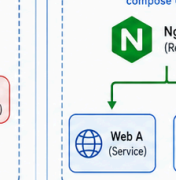
compose 네트워크



• Redis로 세션, 캐시, 카운터 등 처리
• 성능 향상 및 부하 분산

4 Reverse Proxy
Nginx로 두 웹 서비스 라우팅

compose 네트워크



• Nginx가 라우팅/부하분산/SSL 종료
• 두 개의 웹 서비스를 외부에 노출

5 Queue + Worker
비동기 작업 처리 패턴

compose 네트워크



• 메시지 큐로 비동기 작업 처리
• 안정적인 백그라운드 작업 파이프라인

🔄 검증 루프

1 docker compose config

구성 파일 유효성 검사

✓ 실패 시 수정 후 다시

2 docker compose up -d

서비스 배경 실행

✓ 컨테이너 생성 확인

3 docker compose ps

컨테이너 상태 확인

✓ Up(healthy) 확인

4 docker compose logs -f

로그 스트리밍 확인

✓ 예외 여부 확인

5 curl / exec

서비스 응답/내부 실행 검증

✓ 정상 응답 확인

6 docker compose down

자원 정리

✓ 깔끔한 종료 확인

⚠️ 실패 드릴

❗ 포트 충돌
포트 변경 또는 중지

❗ 의존 서비스 미가동
depends_on, healthcheck 확인

❗ 현물/데이터 문제
볼륨 마운트 경로, 권한 확인

❗ 네트워크 문제
같은 네트워크 사용 확인

❗ 헬스체크 실패
로그로 원인 파악 후 재시작

down vs down -v

docker compose down

- ✓ 컨테이너, 네트워크 제거
- ✓ 볼륨(데이터)은 유지

docker compose down -v

- ❌ 컨테이너, 네트워크 제거
- ❌ 익명 볼륨까지 삭제(데이터 삭제)

💡 개발 초기에는 down -v 로 깔끔하게!
중요 데이터가 있다면 down 만 사용!

수업 목표

- 긴 `docker run` 명령이 왜 인수인계와 재현성을 망가뜨리는지 설명한다.
- `compose.yaml`의 `services`, `ports`, `environment`, `volumes`, `networks`, `depends_on`을 읽는다.
- 모든 아키텍처 템플릿에 같은 검증 루프를 적용한다.

왜 Compose인가

Day 3~4에서 실행한 명령은 길고, 순서가 중요하고, 사람이 빠뜨리기 쉽다. Web container를 띄우고, DB를 띄우고, network를 맞추고, env를 넣고, volume을 붙이고, log를 확인하는 과정을 매번 손으로 입력하면 팀원이 같은 환경을 재현하기 어렵다.

Compose는 이 실행 조건을 파일로 남긴다. `compose.yaml`은 단순한 편의 문법이 아니라 local architecture template이다.

docker run에서 하던 일	Compose에서 읽는 위치
image 선택	services.<name>.image 또는 build
host port 공개	ports
runtime config	environment, env_file
data 보존	volumes
service 간 통신	service name, project network
실행 순서 힌트	depends_on

공통 검증 루프

모든 템플릿은 다음 순서로 확인한다.

```
docker compose config
docker compose up -d
docker compose ps
docker compose logs --tail 80
```

서비스별 확인은 architecture마다 다르다.

서비스 유형	확인 예시
Web	curl -I http://localhost:<port>
API	curl -s http://localhost:<port>/<resource>
DB	docker compose exec db psql ...
Cache	docker compose exec redis redis-cli GET ...
Queue/Worker	docker compose logs worker --tail 40
Proxy	proxy port로 접속하고 upstream service port는 직접 열지 않는다

Cleanup 기준

```
docker compose down
# DB/cache data까지 지워도 되는 실습에서만
# docker compose down -v
```

down은 container와 network를 정리한다. down -v는 named volume까지 지운다. DB가 있는 템플릿에서 down -v는 실습 데이터 삭제다.

핵심 포인트

1교시는 문법 암기 시간이 아니다. 이후 2~8교시에서 자주 쓰는 아키텍처를 바로 실행하기 위해 공통 독해법과 공통 검증 루프를 맞추는 시간이다.

Compose file을 읽을 때는 YAML 줄을 외우지 말고 다음 질문으로 읽는다.

외부에서 들어오는 traffic은 어디로 들어오는가?
service끼리는 어떤 이름으로 만나는가?
상태를 가진 service는 어떤 volume을 쓰는가?
실패하면 어떤 logs/exec/curl 결과를 먼저 볼 것인가?
정리할 때 data를 지워도 되는가?